



VR-CLOUD[®] VR/FEM/
CAD/SaaS
Engineer's Studio[®]
UC-win/Road/UC-1/Suite

開発者ガイド

UC-win/Road PythonAPI について

UC-win/Road Ver. 19.0.1

株式会社 フォーラムエイト
東京都港区港南 2-15-1 品川インターシティ A 棟 21F

 **FORUM 8[®]**

Contents

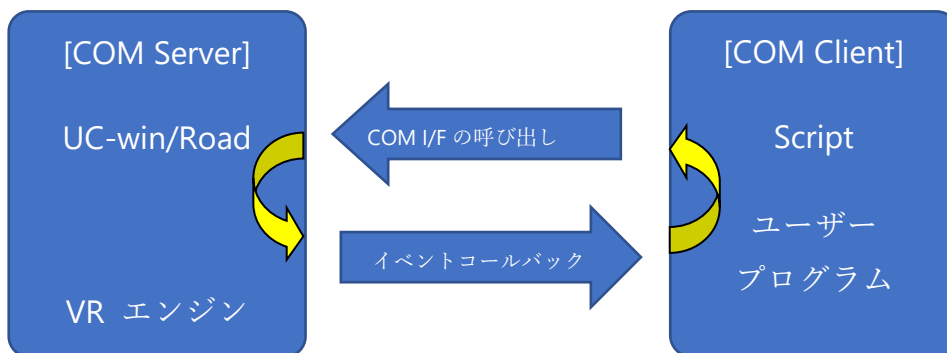
1. UC-win/Road Python API 概要	4
1.1. COM インターフェース	4
1.2. API の構造	4
2. 環境構築	6
2.1. Python のインストール	6
2.2. Python パッケージのインストール	7
2.3. Python スクリプトの格納	8
2.4. ライセンスについて	9
2.5. COM サーバーの登録／解除	9
2.5.1. 登録	9
2.5.2. 解除	10
2.5.3. バージョン切替えについて	10
3. Python スクリプトの実行	11
3.1. UC-win/Road からのスクリプト実行	11
3.1.1. リボンメニュー「Python API」タブ	11
3.1.2. Python スクリプト編集画面からの実行	12
3.1.3. シナリオからのスクリプト実行	13
3.1.4. スクリプトマネージャ画面 (Python Script Manager)	14
3.2. 外部ツールからのスクリプト実行	15
4. Python スクリプトの RD ファイルへの保存	15
5. API の実装方法	16
5.1. COM サーバーへの Dispatch	16
5.1.1. UC-win/Road の ProgID	16
5.1.2. アーリーバインディング	16
5.1.3. レイトバインディング	17
5.2. インターフェースへのアクセス	17
5.2.1. メソッド	17
5.2.2. プロパティ	19
5.2.3. インターフェースの解放	20
5.3. コールバックイベント	20
5.3.1. コールバックイベントのインターフェース	20
5.3.2. コールバックの登録、解除	21
5.3.3. コールバックのタイプ	22

5.3.4. イベントループ	23
5.4. 列挙型の参照	24
6. サンプルスクリプト	25
6.1. Sample_ContextControl	26
6.2. Sample_CoordinateConversion	27
6.3. Sample_GazeTrackReceive	27
6.4. Sample_InputDeviceLog.....	28
6.5. Sample_MainCameraOperation	28
6.6. Sample_OutputVehicleInfo	29
6.7. Sample_OverrideCarInput	29
6.8. Sample_OverrideCarMovement	29
6.9. Sample_RoadInformation	30
6.10. Sample_SimulationScreen	31
6.11. Sample_ThreeDModelCommand.....	31
6.12. Sample_UserVariables.....	32
6.13. Sample_VehiclePlacement.....	33
6.14. Sample_VisualOptionControl	33
6.15. Sample_BrakeAssist	33
6.16. Sample_ObstructiveDriving	34
6.17. Sample_DiagnosisDriving	34
6.18. Sample_VirtualDisplay	35
6.19. Sample GPSroads	35

1. UC-win/Road Python API 概要

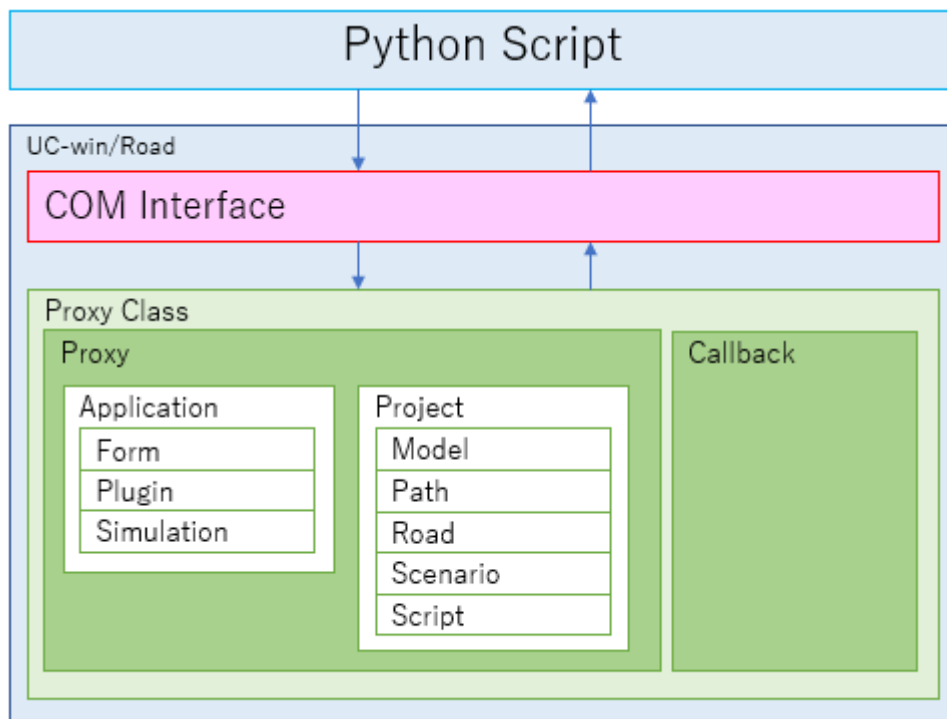
1.1. COM インターフェース

UC-win/Road Python API は COM インターフェースで実装しています。
COM は Component Object Model の略で Microsoft 社によるソフトウェアコンポーネントのバイナリインターフェース規格です。Windows プラットフォーム上で言語に依存しないプロセス間通信のオブジェクト作成を可能にします。COM インターフェースは COM サーバーと COM クライアントの通信を提供します。



1.2. API の構造

C++API と同等機能 (Proxy Class) を COM インターフェースで提供します。



COM インターフェースの詳細は UC-win/Road Python API リファレンス (UCwinRoad_PythonAPI.chm) を参照してください。

UCwinRoad::IUCwinRoadCom が COM のエントリーオブジェクトです。

UCwinRoad PythonAPI 17.00.00

UCwinRoad::IF8ProjectProxy インタフェース

プロジェクトのプロキシクラスです。プロジェクトは保存されるRDファイルに相当します。 [詳解]

UCwinRoad::IF8ProjectProxy の継承関係図

```
graph TD
    IDispatch --> UCwinRoad_IF8ObjectProxy[UCwinRoad::IF8ObjectProxy]
    UCwinRoad_IF8ObjectProxy --> UCwinRoad_IF8ProjectProxy[UCwinRoad::IF8ProjectProxy]
```

公開メンバ関数

HRESULT_stdcall GetActualHeightAt ([in] double X, [in] double Y, [in] IF8COMHeightSearchConditionsType *conditions, [out] double *Height, [out, retval] VARIANT_BOOL *Value)
指定した座標の高さを計算します。

HRESULT_stdcall MakeSpeedlimitPoint ([in] enum F8COMCurveFamilyTypeEnum typeOfCurve, [in] IF8ObjectProxy *curve, [in] double distanceAlongCurve, [out, retval] IF8SpeedlimitPointProxy **Value)
制限速度点を作成します。

HRESULT_stdcall DeleteSpeedlimitPoint ([in] long ID)
制限速度点を削除します。

基盤クラス UCwinRoad::IF8ObjectProxy に属する継承公開メンバ関数

プロパティ

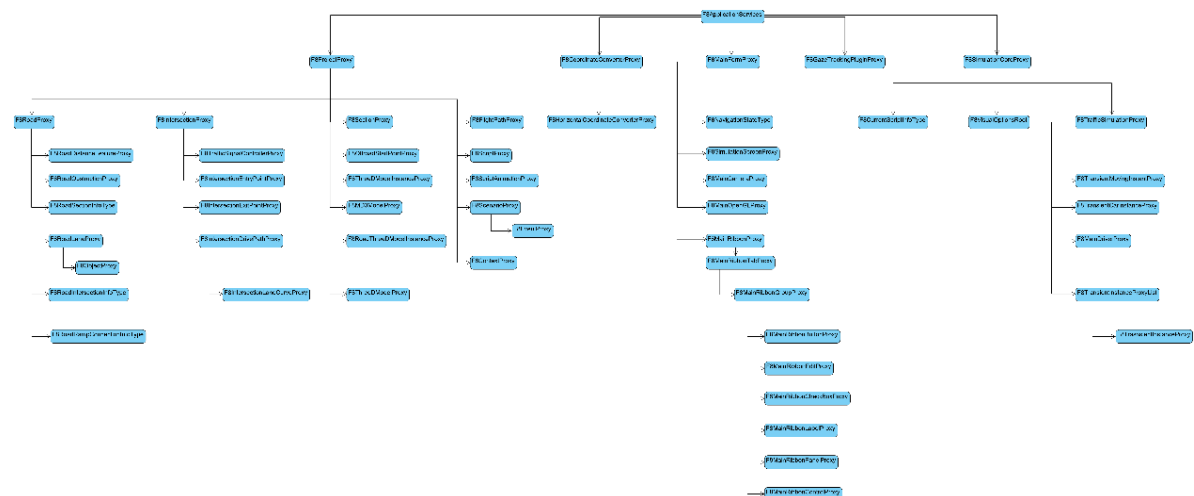
int SectionsCount [get]
プロジェクトが保持する断面設定の数を取得します。

IF8SectionProxy Section ([in] int idx) [get]

Proxy Class は UC-win/Road の主要機能へのアクセスを提供します。

F8AppicationServices がエントリークラスです。

Proxy Class の構造の詳細はファイル **AccessTree.svg** をブラウザで閲覧して下さい。



2. 環境構築

UC-win/Road で Python スクリプトを実行するための環境構築について説明します。

2.1. Python のインストール

以下から Python をインストールして下さい。

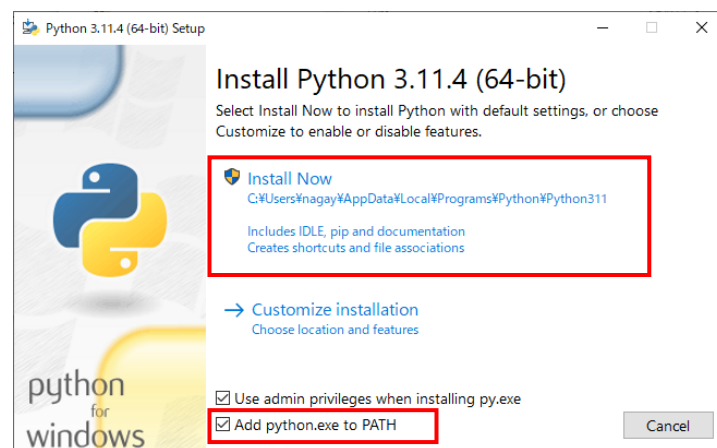
<https://www.python.org/downloads/>

UC-win/Road 19.0.1 は、Python version 3.7 ～3.11 に対応しています。

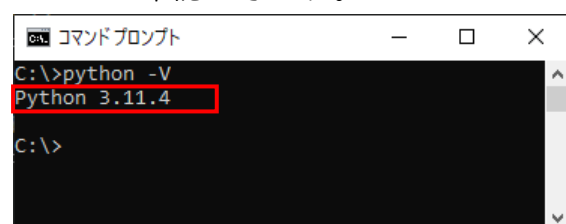
32bit/64bit バージョンは UC-win/Road のバージョンに合わせて下さい。

ヒント：

環境パスへの追加をチェックしてインストールして下さい。

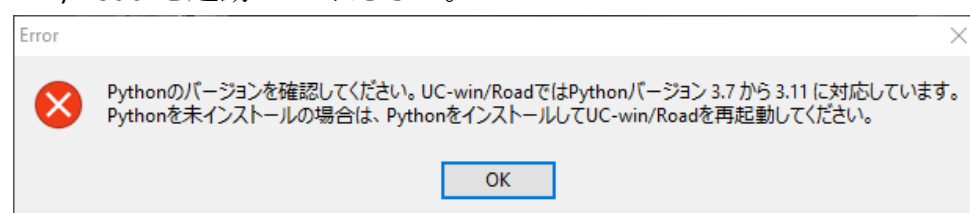


インストール完了後に、コマンドプロンプトで「python -V」で Python のバージョンを確認できます。



ヒント：

Python 対応バージョンを未インストールの PC で、本機能の操作を行った場合は下記メッセージが表示されます。Python をインストールして再度 UC-win/Road を起動してください。



2.2. Python パッケージのインストール

pip という Python パッケージのインストールを行うシステムで、パッケージをインストールします。

パッケージのインストールの詳細については Python ユーザーガイドを参照して下さい。

<https://packaging.python.org/ja/latest/tutorials/installing-packages/>

COM を利用可能とするため、Python パッケージ「pywin32」をインストールします。コマンドプロンプトで以下を実行してください。

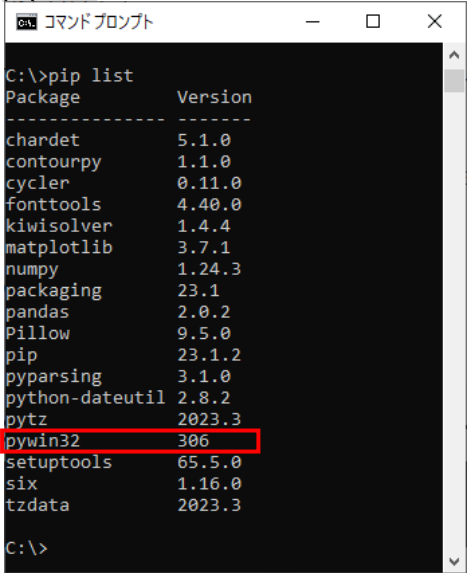
```
pip install pywin32
```

UC-win/Road SDK に同梱しているサンプルを実行するため、サンプルスクリプトで使用している Python パッケージ「pandas」、「matplotlib」、「PyOpenGL」もインストールしてください。コマンドプロンプトで以下を実行してください。

```
pip install pandas  
pip install matplotlib  
pip install PyOpenGL
```

コマンドプロンプトで「pip list」を実行し、インストール済のパッケージの一覧を確認できます。

```
pip list
```



Package	Version
chardet	5.1.0
contourpy	1.1.0
cycler	0.11.0
fonttools	4.40.0
kiwisolver	1.4.4
matplotlib	3.7.1
numpy	1.24.3
packaging	23.1
pandas	2.0.2
Pillow	9.5.0
pip	23.1.2
pyparsing	3.1.0
python-dateutil	2.8.2
pytz	2023.3
pywin32	306
setuptools	65.5.0
six	1.16.0
tzdata	2023.3

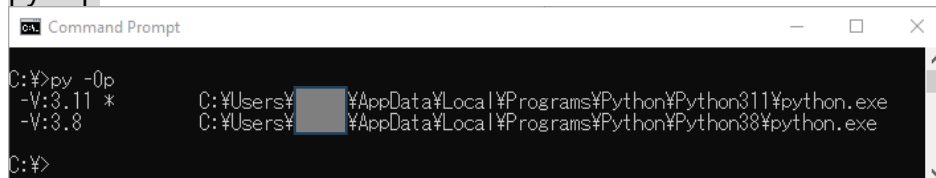
ヒント：

PC 内に複数の Python がインストールされている場合、UC-win/Road は、バージョンが新しい Python を使用します。Python ではバージョンごとにパッケ

ージをインストール必要がありますので、バージョンごとに環境設定を行ってください。

コマンドプロンプトで「py -0p」を実行し、PC 内の Python のすべてのバージョンとそのパスが確認できます。

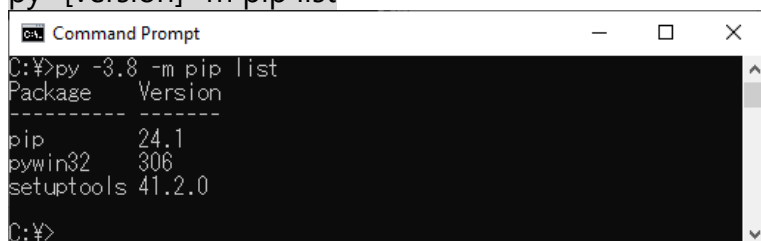
py -0p



```
C:\>py -0p
-V:3.11 *      C:\Users\%user%\AppData\Local\Programs\Python\Python311\python.exe
-V:3.8         C:\Users\%user%\AppData\Local\Programs\Python\Python38\python.exe
C:\>
```

バージョンを指定して、インストール済のパッケージ一覧を確認することができます。

py -[version] -m pip list



```
C:\>py -3.8 -m pip list
Package      Version
-----
pip          24.1
pywin32      306
setuptools   41.2.0
C:\>
```

バージョンを指定して、パッケージをインストールすることができます。

py -[version] -m pip install [package name]

2.3. Python スクリプトの格納

UC-win/Road で使用する Python スクリプトはすべて下記ディレクトリへ格納してください。実行スクリプトと、実行スクリプトへ import する自作のスクリプトも含めてすべて下記ディレクトリへ格納してください。

格納先：

<データディレクトリ>%Plugins%PythonAPIPlugin

<データディレクトリ>はアプリケーションデフォルト画面で設定するユーザデータディレクトリです。

ヒント：

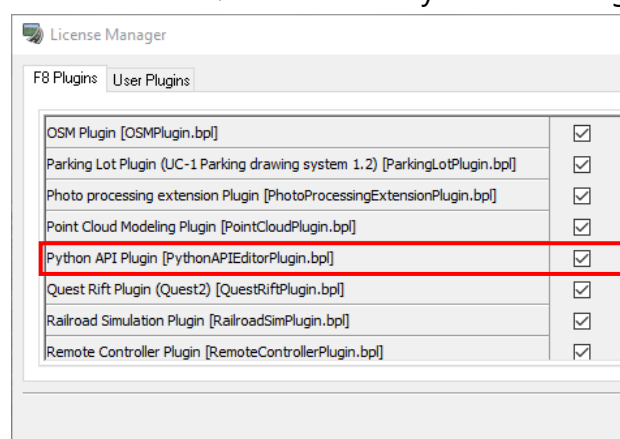
・上記ディレクトリは、UCwinRoad17.2.0 からはインストール時に自動的に生成されます。それ以前のバージョンではユーザーが手作業でディレクトリを作成してください。

・スクリプト格納先ディレクトリを限定するのは、実行スクリプトでモジュールや Class の import を可能とするためです。Python では実行スクリプトにて import するスクリプトの格納パスが Python へ設定されている必要があります。UC-win/Road では起動時に自動的に上記格納ディレクトリのパスを Python へ設定します。

2.4. ライセンスについて

Python API を使用するためには「UC-win/Road SDK(SS)」か「UC-win/Road SDK スイートバンドル(SS)」のライセンスが必要です。

ライセンスマネージャにて Python API Plugin を有効にしてください。



ヒント：

ライセンスがない場合にも、RD ファイル内の Python スクリプトを実行することはできますが、スクリプトの追加、編集等の機能は使用できません。

2.5. COM サーバーの登録／解除

本 API を使用するにあたり、UC-win/Road (COM サーバー) を PC 上のシステムレジストリに登録する必要があります。

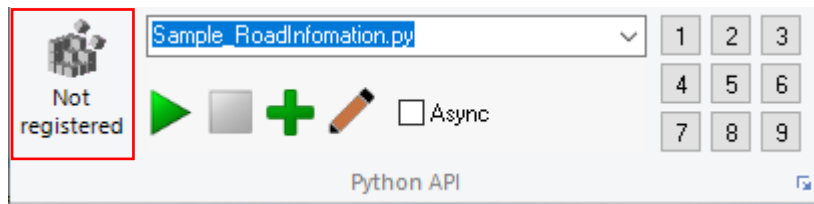
リボンメニュー「Python API」タブからレジストリへの登録、解除を行います。

ヒント：

ライセンスなしの場合は Python スクリプトが保存されている RD ファイルを開いた時のみ「Python API」タブが表示されます。

2.5.1. 登録

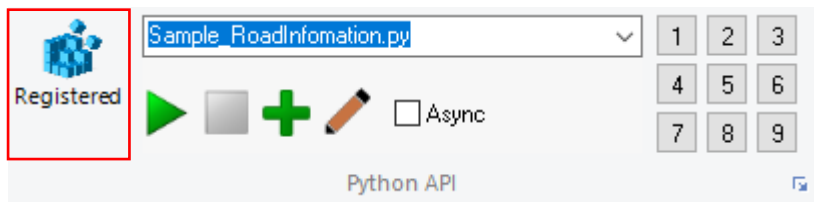
リボンメニューにてレジストリアイコン「Not registered」を押下すると COM サーバーがレジストリへ登録されます。



登録されるとレジストリアイコンが「Registered」へ変わります。

2.5.2. 解除

この状態でレジストリアイコンを押下すると COM サーバーがレジストリからされます。



2.5.3. バージョン切替えについて

UC-win/Road では、1 つの PC に異なるバージョンの COM サーバーの登録を許容していません。UC-win/Road のバージョンアップ等でバージョンを切り替える場合は、現在 COM サーバーを登録している UC-win/Road で COM サーバーの登録解除を行い、その後、切り替えるバージョンの UC-win/Road で COM サーバーの登録を行ってください。

なお、UC-win/Road アンインストール時には、自動的に COM サーバーの登録が解除されます。

COM サーバーには UC-win/Road のバージョンごとにユニークな ProgID を設定しています。

同じバージョンの UC-win/Road で使用する場合は一度 COM サーバーを登録すれば以降の使用時では登録操作は必要ありません。

バージョンを切り替える場合の手順

1. 登録しているバージョンの UC-win/Road で COM サーバーの登録解除を行う。
2. その後、切り替えるバージョンの UC-win/Road で COM サーバーの登録を行う。

ヒント：

バージョン切替え時にスクリプト実行でエラーとなる場合は、サンプルスクリプト「ShowGeneratePath.py」を実行して win32com で生成するキャッシュ情報の格納ディレクトリを開き、ディレクトリ配下のフォルダ、ファイルをすべて削除するとエラーが解消することがあります。キャッシュ情報はスクリプト実行すると再生成されるので削除しても問題ありません。

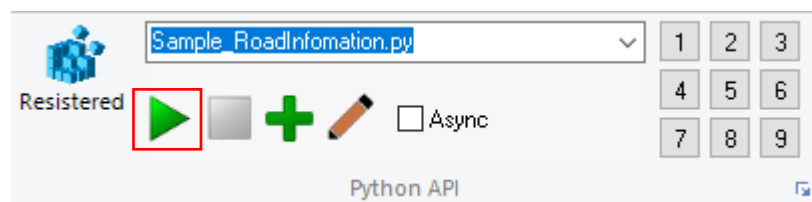
3. Python スクリプトの実行

3.1. UC-win/Road からのスクリプト実行

UC-win/Road での Python スクリプトの実行と関連する UI 操作について説明します。

3.1.1. リボンメニュー「Python API」タブ

リストでスクリプトを選択し、実行ボタンを押下します。



ヒント：

ライセンスが有効の場合はリボンメニューへ「Python API」タブが常時表示されます。

ライセンスが無効の場合は Python スクリプトが保存されている RD ファイルを開いた時のみ「Python API」タブが表示されます。

3.1.1.1. UI 説明

- ・スクリプト選択リスト 
対象スクリプトを選択することができます。
- ・実行ボタン 
選択中のスクリプトを実行します。
- ・停止ボタン 
非同期でのスクリプト実行時にスクリプトの実行を停止します。
- ・追加ボタン 
スクリプトを読み込みます。ボタン押下で読込ダイアログが開きます。
ファイルは複数選択が可能です。
ライセンスが無効の場合は、機能は無効になり使用できません。

- ・編集ボタン 

スクリプト編集画面へ遷移します。

- ・非同期チェックボックス ☐ Async

イベントコールバックを待機するためメッセージループを使用するスクリプトを実行する場合にONにします。ONにするとメインスレッドとは別のスレッドでスクリプトを実行します。

- ・ユーザーフラグボタン 1 ~ 9

IF8ApplicationServicesProxy.PythonScriptUserFlg の ON/OFF を切替します。
ユーザーがスクリプトで利用できる任意のフラグです。

- ・オプションインジケータ 

スクリプトマネージャ画面へ遷移します。

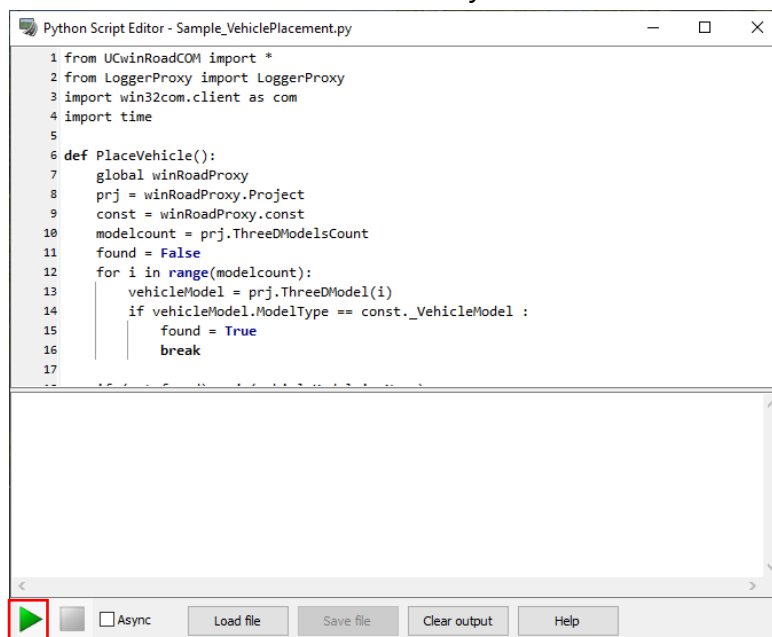
ライセンスが無効の場合は、機能は無効になり使用できません。

3.1.2. Python スクリプト編集画面からの実行

Python スクリプトの編集、実行、読込、保存を行う画面です。

実行ボタンにてスクリプトを実行します。

本画面への遷移方法：リボン [Python API] - [編集ボタン]を押下



3.1.2.1. UI 説明

- ・エディタ部（画面上部）

選択中のスクリプトを表示します。スクリプトを編集することが可能です。

ライセンスが無効の場合は、機能は無効になり使用できません。

- ・結果出力部（画面下部）

スクリプト実行結果を表示します。

非同期チェックを ON とした場合は、スクリプト結果は表示されません。

実行結果はスクリプトからファイル出力等を行い確認してください。

- 実行ボタン

スクリプトを実行します。

- 停止ボタン

非同期でのスクリプト実行時にスクリプトの実行を停止します。

- 非同期チェックボックス ☐ Async

イベントコールバックを待機するためメッセージループを使用するスクリプトを実行する場合に ON にします。

- ファイル読込ボタン

スクリプトを読み込む場合に使用します。

ライセンスが無効の場合は、機能は無効になり使用できません。

- ファイル保存ボタン

スクリプトを保存します。スクリプトを編集するとボタンが有効になります。

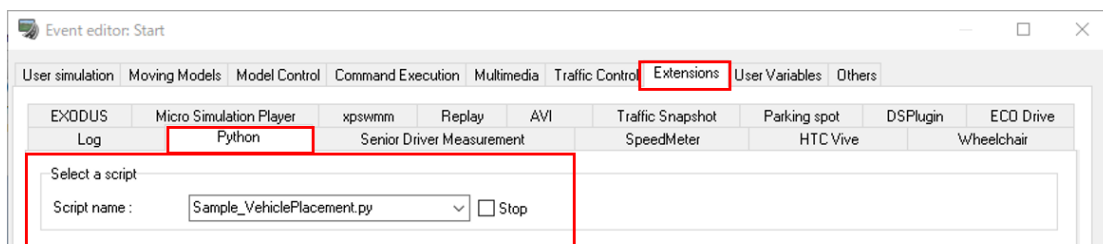
ライセンスが無効の場合は、機能は無効になり使用できません。

- 出力クリアボタン

出力部をクリアします。

3.1.3. シナリオからのスクリプト実行

シナリオのイベントからスクリプトを実行することができます。



イベントの編集画面の「拡張機能」タブで「Python」タブを選択します。

あらかじめ読込んだ Python スクリプトの実行イベント、停止イベントを作成することができます。

- スクリプト実行：

リストでスクリプトを選択し、停止チェックは OFF とします。

イベント実行時に選択したスクリプトが実行されます。

スクリプトは非同期チェック ON（リボン UI、スクリプト編集画面にある

チェックボックス) として実行されます。

- ・ スクリプト停止 :

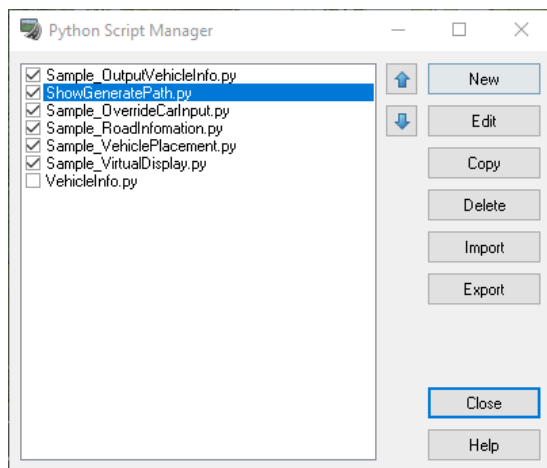
停止チェック ON とします。

実行イベントの場合は OFF に設定して下さい。

3.1.4. スクリプトマネージャ画面 (Python Script Manager)

Python スクリプトの並べ替え、追加、編集、コピー、削除、インポート、エクスポート、リボンメニューのスクリプト選択リストへ表示するスクリプトの選択切替を行う管理画面です。

本画面への遷移方法 : リボン [Python API] - [オプションインジケータ] 押下



3.1.4.1. UI 説明

- ・ [追加] ボタン

新規にスクリプトを追加します。

画面操作については、「Python スクリプト編集」画面を参照してください。

- ・ [編集] ボタン

選択したスクリプトを編集します。

画面操作については、「Python スクリプト編集」画面を参照してください。

- ・ [複写] ボタン

選択したスクリプトを複写します。

- ・ [削除] ボタン

選択したスクリプトを削除します。

シナリオのイベントへスクリプトを設定している場合は削除できません。

- ・ [読込(Import)] ボタン

スクリプトを読み込みます。ボタン押下で読込ダイアログが開きます。

ファイルは複数選択が可能です。

- ・ [出力(Export)]ボタン

選択したスクリプトを Python スクリプト (*.py) としてファイルに出力します。

このボタンをクリックすると、「フォルダの選択」ダイアログが開きます。

出力先のフォルダを選択し、ファイルを出力して下さい。

- ・ [閉じる]ボタン

スクリプトマネージャ画面を閉じます。

- ・ チェックボックス

リボンのスクリプト選択リストへ表示するスクリプト選択切替をします。

チェックしたスクリプトのみリボンメニューのリストで表示します。

シナリオのイベントへスクリプトを設定している場合は非表示にできません。

3.2. 外部ツールからのスクリプト実行

Visual Studio Code などの外部ツールからもスクリプトを実行することができます。COM サーバー登録した UC-win/Road を起動し、外部ツールにてスクリプトを実行してください。外部ツールにてデバッグ実行も可能です。

注意点

1. 実行速度は UC-win/Road での実行と比べて遅くなります。
2. スクリプトでメッセージループをしている場合に、ループを終了させるためのトリガーとして、リボンUIのユーザーフラグボタンをご利用下さい。

例：実行前にユーザーフラグON、停止時にユーザーフラグOFF

スクリプトにてユーザーフラグのOFFを判定してループを終了

3. ライセンスが無効の場合は外部ツールからのスクリプト実行はできません。

4. Python スクリプトの RD ファイルへの保存

UC-win/Road に読み込んだ Python スクリプトはプロジェクト保存時に RD ファイルへ保存されます。保存した RD ファイルを開くと、Python スクリプトが読み込まれた状態でプロジェクトが開始されます。ライセンスが無効の場合でも RD ファイル内のスクリプトを実行することができます。

5. API の実装方法

この章では、API を用いた Python スクリプトの実装方法について説明します。

5.1. COM サーバーへの Dispatch

Python から UC-win/Road (COM サーバー) へ接続するために ProgID を用いて Dispatch を行います。

Dispatch の方法にはアーリーバインディングとレイトバインディングの2種類があります。アーリーバインディングはコンパイル時に静的にバインディングする方法で、レイトバインディングは実行時に動的にバインディングする方法です。

ヒント：

サンプルスクリプト「UCwinRoadCOM.py」で COM サーバーへの Dispatch をしていますので参照してください。

サンプルスクリプト「ShowGeneratePath.py」は、win32com のアーリーバインディングで作成されるキャッシュ情報の格納ディレクトリを開きます。必要に応じて実行して確認してください。

5.1.1. UC-win/Road の ProgID

UC-win/Road のバージョン毎に ProgID はユニークに設定しています。

```
"UCwinRoad.UCwinRoadCom_●●●●"
```

●●●●：バージョン毎に設定される番号

1, 2 桁：メジャーバージョン

3 桁目：マイナーバージョン

4 桁目：パッチバージョン

例：UCwinRoad Ver19.0.1 の場合

```
"UCwinRoad.UCwinRoadCom_1901"
```

5.1.2. アーリーバインディング

アーリーバインディングを行うとプログラム実行前にオブジェクトのメソッド、プロパティ、定数や列挙型についてキャッシュ情報が作られます。

UC-win/Road の Python API ではアーリーバインディングを推奨します。

以下では強制的にアーリーバインディングが行われます。

```
import win32com.client as com
winRoad = com.gencache.EnsureDispatch("UCwinRoad.UCwinRoadCom_1901")
```


キャッシュ情報がない場合に、下記フォルダにバインディングされたタイプライブラリとインターフェースのキャッシュが出力されます。

%LOCALAPPDATA%\Temp\gen_py\%xx%\CLSID

xx: Python バージョン

CLSID: COM サーバーのタイプライブラリの GUID

以下で出力先を確認することができます。

```
dir_name = com.gencache.GeneratePath()
```

5.1.3. レイトバインディング

キャッシュ情報がある場合はアーリーバインディングを使用し、そうでない場合はレイトバインディングを行います。

```
winRoad = com.Dispatch("UCwinRoad.UCwinRoadCom_1901")
```

強制的にレイトバインディングを行います。

```
winRoad = com.dynamic.Dispatch("UCwinRoad.UCwinRoadCom_1901")
```

5.2. インターフェースへのアクセス

UC-win/Road (COM サーバー) へ Dispatch して IUCwinRoadCom インターフェースを取得します。このインターフェースから API のエントリーポイントである IF8ApplicationServicesProxy インターフェースを取得します。

```
app = winRoad.ApplicationServices
```

同様に IF8ApplicationServicesProxy インターフェースから各インターフェースへアクセスします。

例: IF8ApplicationServicesProxy インターフェース から IF8ProjectProxy インターフェースを取得

```
proj = app.Project
```

インターフェースへアクセスする順、方法は、UC-win/Road Python API リファレンス (UCwinRoad_PythonAPI.chm) と Proxy Class の構造詳細ファイル **AccessTree.svg** を参照してください。

5.2.1. メソッド

インターフェースのメソッドコールについて説明します。

以下のようにインターフェースのメソッドをコールします。

例: IF8RoadProxy インターフェース GetPositionAt ()

```
HRESULT _stdcall GetPositionAt  
([in] double Distance,  
[out, retval] IF8COMdVec3 **Value)
```

```
for i in range(count):
```

```

road = prj.Road(i)
roadLength = road.Length
dist = 0.0
while dist <= roadLength:
    roadPos = road.GetPositionAt(dist);

```

注意点

1. メソッドコール時にインターフェース型の Out パラメータを設定する場合は以下の様に Dispatch で取得したインターフェースを設定します。Dispatch に使用する ProgID は 'UCwinroad.XXX' です。XXX はインターフェース名から接頭辞の "I" を除いたシンボル名として下さい。

```

srcVec2 = com.Dispatch('UCwinRoad.F8COMdVec2')
dstVec2 = com.Dispatch('UCwinRoad.F8COMdVec2')
convRes = com.Dispatch('UCwinRoad.F8COMHcsConvertResultType')
hConverter.Convert(const._hcLocal_XY, const._hcWGS84_LonLat,
srcVec2, dstVec2, convRes);

```

2. Python では参照渡しを概念をサポートしていないため、単純なパラメータを参照渡しすることができません。

Out パラメータの型がインターフェース型以外の単純なパラメータの場合、Out パラメータが更新されず戻り値として返却されます。

以下のように Out パラメータを戻り値として記述して値を取得して下さい。

例：IF8ProjectProxy インターフェース GetActualHeightAt ()

```

HRESULT _stdcall GetActualHeightAt
([in] double X,
[in] double Y,
[in] IF8COMHeightSearchConditionsType *conditions,
[out] double *Height,
[out, retval] VARIANT_BOOL *Value)

```

以下記述では out パラメータの h が更新されません。

```

prj.GetActualHeightAt(100, 500, a, h)

```

以下の様に戻り値で取得する記述をすると h が更新されます。

retval, out パラメータの順に記述します。

out パラメータが複数ある場合は左から順に記述して下さい。

```

ret, h = prj.GetActualHeightAt(100, 500, a, h)

```

5.2.2. プロパティ

インターフェースのプロパティについて説明します。

5.2.2.1. Getter

以下の様にプロパティを取得することができます。

例：IF8RoadProxy インターフェース Name、Length プロパティ

```
BSTR  Name[get]
double Length[get]

for i in range(count):
    road = prj.Road(i)
    name = road.Name
    roadLength = road.Length
```

5.2.2.2. Setter

以下の様にプロパティを設定することができます。

例：IF8MainFormProxy インターフェース NavigationState プロパティ

```
IF8NavigationStateType  NavigationState[set]

naviSt = mainForm.InitialNavigationState
mainForm.NavigationState = naviSt #set
```

注意点

1. 引数を持つプロパティのシンボル名

win32com.client.gencache.EnsureDispatch でアーリーバインディングして作成されたキャッシュ情報では、引数を持つプロパティの **Setter** はメソッドとして変換されます。UC-win/Road Python API リファレンス（UCwinRoad_PythonAPI.chm）に記載しているシンボル名とは異なるシンボル名に変換されるので注意してください。

例：IF8SimulationCoreProxy インターフェース UserVariable プロパティ

```
double  UserVariable ([in] int idx)[get, set]
```

インデックスを引数に持つプロパティ

UserVariable→**SetUserVariable** とシンボル名称の先頭に **Set** が付与されます。

Setter を使う場合はシンボル名の先頭に **Set** を付与して使用して下さい。

```
simCore = app.SimulationCore
#getter
v = simCore.UserVariable(0)
#setter
simCore.SetUserVariable(0, 88)
```

5.2.3. インターフェースの解放

取得したインターフェースは使用後に明示的に `None` を代入して解放して下さい。

```
proj = None
app = None
winRoad = None
```

5.3. コールバックイベント

COM クライアントで COM サーバーからイベントを受信する方法について説明します。

5.3.1. コールバックイベントのインターフェース

IF8ApplicationServicesProxyEvents の様に、インターフェースの末尾に **Events** が付与されたインターフェースに、コールバックイベントインターフェースが用意されています。また、IF8ObjectProxyEvents が設定されたインターフェースも存在します。イベントインターフェースについては、UC-win/Road Python API リファレンス（UCwinRoad_PythonAPI.chm）の詳細をご参照ください。

例：

UCwinRoad::IF8ApplicationServicesProxyEvents インタフェース

F8ApplicationServicesProxyオブジェクトのイベントインターフェース。[詳解]

公開メンバ関数

HRESULT	OnPluginAbleMenus ([in] VARIANT_BOOL enable)
	アプリケーションがリボンメニューの有効・無効状態を切り替えるときに呼び出されるコールバック。
HRESULT	OnErrorOrWarning ([in] enum F8COMApplicationErrorTypeEnum errorType, [in] unsigned short errorCode, [in] BSTR errorMessage)
	エラー、警告、または情報をユーザーに通知する時に呼び出されるコールバック。
HRESULT	OnNewProject (void)
	プロジェクトを新規作成した時、または開いた時に呼び出されるコールバック。
HRESULT	OnBeforeSaveProject ([in] BSTR Name)
	プロジェクトの保存前に呼び出されるコールバック。
HRESULT	OnAfterSaveProject (void)
	プロジェクトの保存後に呼び出されるコールバック。
HRESULT	OnCloseProjectQuery ([out, retval] VARIANT_BOOL *allow)
	プロジェクトを閉じる前に確認を行うコールバック。
HRESULT	OnBeforeDestroyProject (void)
	プロジェクトを解放する前に行うコールバック。
HRESULT	OnAfterLoadProject (void)
	プロジェクトを読み込んだ後に呼び出されるコールバック。
HRESULT	OnIsExistEventHandler ([in] BSTR funcname, [out, retval] VARIANT_BOOL *Value)
	イベントハンドラクラスにコールバック関数が存在するか確認を行うコールバック。

5.3.2. コールバックの登録、解除

Python スクリプトで COM サーバーへコールバックを登録する方法について説明します。

5.3.2.1. イベントハンドラクラスの定義

例：イベントハンドラクラス ApplicationServicesHandler の定義

```
class ApplicationServicesHandler:
    def SetCOMEventClass(self, events):
        self.events = events
    def OnIsExistEventHandler(self, funcname):
        try:
            func = getattr(self.events, funcname)
        except AttributeError:
            return False
        return True
    def OnPluginAbleMenus(self, enable):
    def OnErrorOrWarning(self, errorType, errorCode, errorMessage):
    def OnNewProject(self):
    def OnBeforeSaveProject(self, name):
    def OnAfterSaveProject(self):
    def OnCloseProjectQuery(self, allow=pythoncom.Missing):
    def OnBeforeDestroyProject(self):
    def OnAfterLoadProject(self):
```

説明

def OnPluginAbleMenus ～ def OnAfterLoadProject(self):

必要なイベントのコールバック関数を実装して下さい。

def SetCOMEventClass(self, events):

def OnIsExistEventHandler(self, funcname):

これらの関数は、COM クライアントから COM サーバーへ、コールバックイベントのハンドラ関数登録を行った際に、COM サーバーからコールバック関数の存在有無を確認するために使用されます。COM サーバーはコールバック関数が存在する場合にのみハンドラ関数登録処理を実施します。

上記 2 関数はサンプルをコピーして必ず実装して下さい。

5.3.2.2. インターフェースへのハンドラ登録

以下の手順で登録します。

1. win32com.client.WithEvents()で対象のインターフェースにコールバック用のイベントハンドラクラスを設定

2. SetCOMEventClass で win32com.client.COMEventClass へ自身の Event 情報を設定
3. 対象のインターフェースの RegisterEventHandlers()でコールバックイベントのハンドラ関数の登録を実施

例：IF8ApplicationServicesProxy にイベントハンドラクラス
ApplicationServicesHandler を設定

```
import win32com.client as com
app = winRoad.ApplicationServices
appEvent = com.WithEvents(app, ApplicationServicesHandler)
appEvent.SetCOMEventClass(appEvent)
app.RegisterEventHandlers()
```

5.3.2.3. インターフェースからハンドラ登録解除

以下の手順でハンドラ登録を解除します。

1. win32com.client.COMEventClass.close ()で、コールバックイベントのハンドラクラスの登録を解除します。
2. 対象インターフェースの UnRegisterEventHandlers()で、コールバックイベントのハンドラ関数の登録を解除します。

```
appEvent.close()
app.UnRegisterEventHandlers()
```

5.3.3. コールバックのタイプ

UC-win/Road のコールバック登録関数には2つのタイプ（通常、ユニーク）があります。通常タイプではイベントに対して複数のコールバックを登録できます。ユニークタイプではイベントに対して1つのコールバックのみを登録できるものです。

コールバックのタイプがユニークの場合、既にコールバック関数が登録されているかを確認して、既に登録されている場合は新しい関数を登録する前に登録解除を呼び出す必要があります。通常のコールバックの場合、確認する必要はありません。

ユニークなタイプでは対象インターフェースに以下の関数を持っています。

- ・ユニークなコールバック関数

登録確認：“IsSetCallback”で始まります。

登録解除：“UnsetCallback”で始まります。

例：IF8TransientInstanceProxy インターフェース
IsSetCallbackOnCalculateMovement()

コールバックの登録、解除を行う場合は、通常とユニークで同じ関数をコールします。

登録：**RegisterEventHandlers()**

解除：**UnRegisterEventHandlers()**

ユニークなタイプで既にコールバック関数が登録されている場合、RegisterEventHandlers()を呼び出すと更新され、UnRegisterEventHandlers()を呼び出すと解除されます。

5.3.4. イベントループ

COM サーバーからのコールバックを受けるためにメッセージループを実装して下さい。

例：メッセージループ（ユーザーフラグによるループ制御）

```
loopFlg = True
app.IsPythonScriptRun = loopFlg
while loopFlg:
    time.sleep(0.0005)
    loopFlg = app.PythonScriptUserFlg(0)
    if loopFlg == False:
        logger.info("loopFlg={}".format(loopFlg))
        logger.info("Script close")

app.IsPythonScriptRun = loopFlg
```

5.3.4.1. スクリプト停止について注意点

イベントループを正しく終了させるためには、Python からスクリプトを終了させる必要があります。Python スクリプトからループを停止し、イベントハンドラの登録解除処理を行わない場合、コールバックイベントが継続することになりますのでご注意ください。

以下 2 通りの方法を提供しています。

- ・ UC-win/Road のリボン UI、スクリプトエディタ上の停止ボタン

IF8ApplicationServicesProxy のプロパティ IsPythonScriptRun（Python スクリプトの実行状態）の ON/OFF を参照してイベントループを終了させて下さい。

- ・ UC-win/Road のリボン UI 上のユーザーフラグボタン
IF8ApplicationServicesProxy のプロパティ PythonScriptUserFlg（ユーザーフラグ）の ON/OFF を参照してイベントループを終了させて下さい。

5.4. 列挙型の参照

win32com.client.constans を利用して COM サーバーの列挙型を参照することができます。これには COM サーバーへアーリーバインディングしておく必要があります。

win32.client.constans をコールして COM 定数のコンテナを生成

```
const = win32com.client.constants
```

例：F8COMTransientTypeEnum._TransientCar の使用例

```
if const._TransientCar == transientType :
```

Python API で使用しているすべての列挙型は 0 から始まる連続的な値で定義されています。

6. サンプルスクリプト

UC-win/Road SDK には Python API を使用する方々の参考になるようにサンプルスクリプトを Python¥ Samples 以下に用意しています。

サンプルスクリプトを実行するためには、環境構築の章を参照して環境設定を行ってください。

No.	ファイル名	概要
1	Sample_ContextControl.py	コンテキスト切り替え
2	Sample_CoordinateConversion.py	座標変換
3	Sample_GazeTrackReceive.py	視線計測連携機能で受信、計算データを受け取る
4	Sample_InputDeviceLog.py	マウス、キーボード、ジョイスティックの入力を受け取る
5	Sample_MainCameraOperation.py	メインカメラの視点位置、視線方向を指定する
6	Sample_OutputVehicleInfo.py	カメラ位置周辺の交通車両の情報を取得し、ファイル出力する
7	Sample_OverrideCarInput.py	運転車両のステアリング入力、アクセル・ブレーキ・クラッチのペダル入力を上書きする
8	Sample_OverrideCarMovement.py	運転車両の挙動計算を上書きし、車両の姿勢情報を与える
9	Sample_RoadInformation.py	道路の情報を取得し、ファイル出力する
10	Sample_SimulationScreen.py	メイン画面のシミュレーションスクリーン設定を変更し、画面の画角や表示位置を変更する
11	Sample_ThreeDModelCommand.py	配置 3D モデルに対し、可動設定のコマンドを送信する
12	Sample_UserVariables.py	ユーザー変数を取得、変更する
13	Sample_VehiclePlacement.py	シミュレーション上に新しい車両オブジェクトを生成する
14	Sample_VisualOptionControl.py	描画オプションを取得、制御する
15	Sample_BrakeAssist.py	運転車両のアクセル・ブレーキのペダル入力を操作して簡易的なアシストブレーキ動作をする
16	Sample_ObstructiveDriving.py	運転車両の前方車両のステアリング入力、アクセル・ブレーキのペダル入力を操作し、簡易的な妨害運転動作を行う
17	Sample_DiagnosisDriving.py	運転車両の簡易的な運転診断をする
18	Sample_VirtualDisplay.py	運転車両の速度に応じてバーチャルディスプレイへ OpenGL でダイレクト描画する

-	UCwinRoadCOM.py	COM サーバーアクセス用 Class のサンプル
-	UCwinRoadUtils.py	Utils 関数サンプル
-	CallbackHandlers.py	コールバックイベントのハンドラクラスのベースクラス、各インターフェースクラスのイベントクラスのひな型のサンプル
-	LoggerProxy.py	ログ出力のためのサンプル
-	VehicleInfo.py	Sample_OutputVehicleInfo.py で使用する Dataclass サンプル
-	ShowCurrentPath.py	カレントディレクトリを開くサンプル
-	ShowGeneratePath.py	win32com のアーリーバインディングで作成されるキャッシュ情報の格納ディレクトリを開くサンプル

6.1. Sample_ContextControl

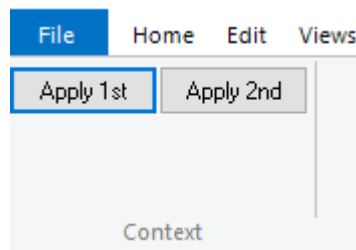
Sample_ContextControl はコンテキストを切り替える動作を試すことが可能なスクリプトです。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「Context」が追加されます。

ボタン「Apply 1st」をクリックすると、プロジェクトに登録された先頭のコンテキストを適用し、コンテキストで決められた設定値が描画オプションなどに適用されます。

ボタン「Apply 2nd」をクリックすると、プロジェクトに登録された 2 つ目のコンテキストを適用し、コンテキストで決められた設定値が描画オプションなどに適用されます。

プロジェクトにコンテキストが存在しない場合は変化しないため、Mt.Parkway などのサンプルデータを読み込んでお試しください。

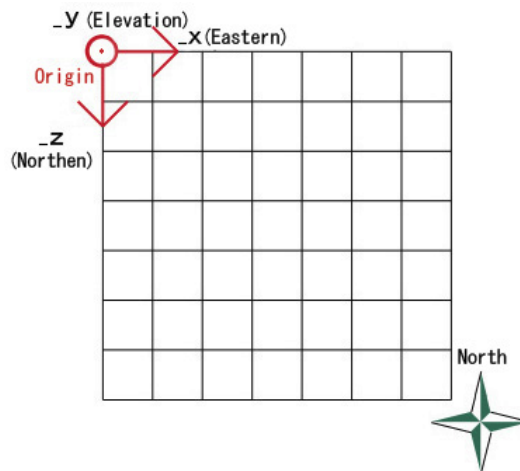


コンテキストは描画オプションやシミュレーション環境を一度に切り替えるために使用する設定です。作成方法や使用方法については UC-win/Road のヘルプや操作ガイダンスをご覧ください。

6.2. Sample_CoordinateConversion

Sample_CoordinateConversion は座標変換機能を試すことが可能なスクリプトです。

API で扱う座標系は下のような OpenGL 座標系になっています。



図：OpenGL 座標系

この OpenGL 座標系や、プログラム上で表示しているローカル座標系やグローバル座標系、また経緯度といった各座標系を相互変換するための機能を座標コンバーターとして提供しています。

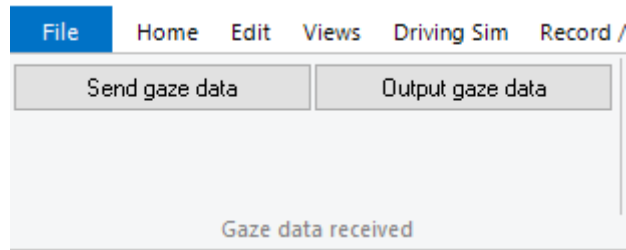
実行すると、単一のローカル座標変換結果と、複数のローカル座標変換結果をスクリプトエディタ画面の結果出力欄とログファイルに出力します。（カレントディレクトリへ出力されます）

OpenGL 座標値やローカル座標値、グローバル座標値を経緯度に変換するためには座標系がプロジェクトに設定されている必要がありますが、プロジェクト作成時に座標系が割り当てられます。

6.3. Sample_GazeTrackReceive

Sample_GazeTrackReceive は視線計測連携機能で受信、計算されたデータを受け取る動作を確認できるスクリプトです。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「Gaze data received」が追加されます。



視線計測連携プラグインが有効となっており、UDP 通信などで視線計測データを受信している状態で、ボタン「Send gaze data」をクリックし、「Output gaze data」をクリックすると受信したデータと計算された値を CSV ファイルへ出力します。

視線計測連携プラグインの詳細については、UC-win/Road ヘルプをご覧ください。

6.4. Sample_InputDeviceLog

Sample_InputDeviceLog はマウス、キーボード、ジョイスティックの入力を受け取るスクリプトです。

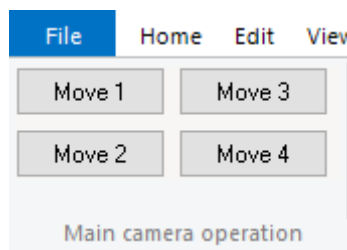
非同期チェックボックスを ON にして、実行すると、マウス、キーボード、ジョイスティックの入力を受け取り、ログファイルを出力します。

(カレントディレクトリへ出力されます)

6.5. Sample_MainCameraOperation

Sample_MainCameraOperation はメインカメラの視点位置、視線方向を指定するサンプルのスクリプトです。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「Main camera operation」が追加されます。



ボタン「Move 1」をクリックすると、視点位置、視線先の位置と傾きの角度を使用してカメラの方向を設定します。

ボタン「Move 2」をクリックすると、視点位置、視線先の位置と上方向ベクトルを使用してカメラの方向を設定します。

ボタン「Move 3」をクリックすると、カメラの姿勢行列を使用してカメラの方向を設定します。

ボタン「Move 4」をクリックすると、モデルビュー行列（カメラ姿勢の逆行列）を使用してカメラの方向を設定します。

なお、行列を用いた場合は UC-win/Road のカメラのパラメータに適用できる形で適用します。

たとえば、X 軸、Y 軸、Z 軸が垂直に交わらない場合姿勢の行列や、スケールを変更するような行列を指定した場合は、そのまま反映されません。

6.6. Sample_OutputVehicleInfo

Sample_OutputVehicleInfo はカメラ位置周辺の交通車両の情報を取得し、ファイル出力を行うスクリプトです。

実行すると、カメラ位置周辺 100m の交通車両を取得し、車両の様々な情報をスクリプトエディタ画面の結果出力欄と CSV ファイルへ出力します。（カレントディレクトリへ出力されます）

	A	B	C	D	E	F	G	H	I	J	K
1	Time stamp	ID	Description	Position X	Position Y	Position Z	Yaw angle	Pitch angle	Roll angle	Direction X	Direction Y
2	47.17.4	32752782	Car coupe	8545.964	273.8677	12225.96	1.301261	-0.00912	0.001381	0.963855	0.009114
3	47.17.4	32752783	Car coupe	8521.056	273.6497	12221.16	1.458594	-0.0081	-0.00074	0.993679	0.008096
4	47.17.4	32752784	Car coupe	8504.335	273.4829	12219.11	1.454315	-0.01049	-0.01715	0.993169	0.010488
5	47.17.4	32752785	Car coupe	8440.284	273.1714	12222.6	-1.24236	0.005167	0.000081	-0.94653	-0.00517
6	47.17.4	32752786	Car coupe	8471.725	273.3362	12219.17	4.595261	0.005066	-1E-06	-0.99314	-0.00507
7											
8											
9											

6.7. Sample_OverrideCarInput

Sample_OverrideCarInput は運転車両のステアリング入力、アクセル・ブレーキ・クラッチのペダル入力を上書きする操作を行うスクリプトです。

運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行すると、運転車両の入力量の上書きを行います。

ステアリング入力、アクセル・ブレーキ・クラッチのペダル入力について上書きするため、これらの入力はキーボードなどから操作できなくなります。

このサンプルではあくまで入力の上書きを行います。

車両計算の結果を上書きして、車両位置等を直接与えたい場合は、Sample_OverrideCarMovement サンプルを確認してください。

6.8. Sample_OverrideCarMovement

Sample_OverrideCarMovement は運転車両の挙動計算を上書きし、車両の姿勢情報を与えるスクリプトです。

このプラグインを試す前に下記の手順でプロジェクトを作成してください。

1. 新規デフォルトプロジェクトを作成します。
2. 道路平面図を開き、道路の作成を選択し、地形上の 2 点をクリックし、道路の作成終了を選択します。
3. 始点の方向変化点を編集し、ローカル座標を X=4900、Y=5000 に設定して確定します。
4. 終点の方向変化点を編集し、ローカル座標を X=5100、Y=5000 に設定して確定します。
5. 道路の縦断編集画面を開き、始点の縦断変化点の編集画面を開き、高さを 560m に設定して確定します。
6. 終点の縦断変化点の編集画面を開き、勾配を 0% に設定して確定します。
7. 道路平面図を確定で閉じます。

運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行すると、クリックすると、運転車両の挙動計算を上書きします。

車両の座標、速度、回転角、方向、車体の方向、エンジン回転数などを上書きして、ジグザグの挙動となるように制御します。

このサンプルでは挙動計算の上書きを行います。

車両の入力値を上書きして、UC-win/Road 側で挙動計算を行う場合は、Sample_OverrideCarInput サンプルを確認してください。

6.9. Sample_RoadInformation

Sample_RoadInformation は道路の情報を取得し、ファイル出力するスクリプトです。

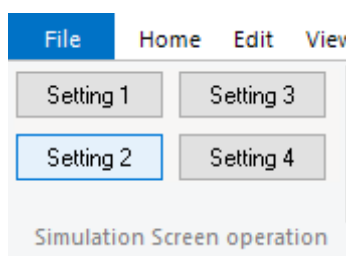
実行すると、プロジェクトの道路を取得し、150m ピッチでの道路の座標、接線方向、曲率、高さ、勾配を求め CSV ファイルを出力します。

	A	B	C	D	E	F	G	H	I	J
1		Name	Length	Distance	Pos X	Pos Z	Direction	Curvature	Height	Slope
2		0 Nihondaira Park Way	1935.578	0	8580.92	12229.95	276.6694	INF	274.23	-1.088
3		1 Nihondaira Park Way	1935.578	150	8432.941	12223.8	241.2964	-60	273.135	-0.499
4		2 Nihondaira Park Way	1935.578	300	8434.858	12354.7	152.5058	INF	271.2889	-1.588
5		3 Nihondaira Park Way	1935.578	450	8406.257	12455.56	293.0124	40.6	265.6421	-8.27529
6		4 Nihondaira Park Way	1935.578	600	8329.973	12543.27	157.8223	-65	253.3872	-7.56674
7		5 Nihondaira Park Way	1935.578	750	8352.215	12672.32	232.1236	60	241.1352	-6.00947
8		6 Nihondaira Park Way	1935.578	900	8219.878	12742.83	242.3144	INF	241.1165	1.063317
9		7 Nihondaira Park Way	1935.578	1050	8087.657	12735.96	307.4846	INF	234.677	-7.68359
10		8 Nihondaira Park Way	1935.578	1200	7948.251	12694.51	275.1296	INF	223.4203	-6.72847
11		9 Nihondaira Park Way	1935.578	1350	7872.668	12597.17	329.6563	-31.2	213.4216	-6.665
12		10 Nihondaira Park Way	1935.578	1500	7805.383	12680.82	181.3717	-50	203.4241	-6.665
13		11 Nihondaira Park Way	1935.578	1650	7860.613	12819.64	156.5292	INF	196.8588	-0.29626
14		12 Nihondaira Park Way	1935.578	1800	7957.275	12927.61	139.7479	28.5	195.8733	-0.86589
15		13 Nihondaira Park Way	1935.578	1935.578	7866.59	12958.69	283.586	INF	194.4361	-1.132

6.10.Sample_SimulationScreen

Sample_SimulationScreen はメイン画面のシミュレーションスクリーン設定を変更し、画面の画角や表示位置を変更するスクリプトです。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「Simulation Screen operation」が追加されます。



ボタン「Setting 1」をクリックすると、ウィンドウサイズによるシミュレーションスクリーンの設定を行います。

ボタン「Setting 2」をクリックすると、実サイズによるシミュレーションスクリーンの設定を行います。

ボタン「Setting 3」をクリックすると、直接 FOV によるシミュレーションスクリーンの設定を行います。

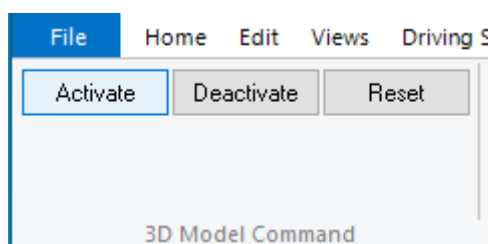
ボタン「Setting 4」をクリックすると、シミュレーションスクリーンの設定を取得し、CSV ファイルへ出力します。

6.11.Sample_ThreeDModelCommand

Sample_ThreeDModelCommand は配置 3D モデルに対し、可動設定のコマンドを送信するスクリプトです。

このプラグインを試す場合は、サンプルプロジェクト Highway.rd を読み込み、リボンメニューのホーム>カメラ位置でコンボボックスから「4D シミュレーション」を選択して、カメラをクレーンの位置に移動してください。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「3D Model Command」が追加されます。



ボタン「Activate」をクリックすると、プロジェクトのすべての配置モデルに対し、コマンド「right」を送信します。

それによって、カメラ位置「4D シミュレーション」の近くにあるクレーンが右回転を行います。

回転中にボタン「Deactivate」をクリックすると、プロジェクトのすべての配置モデルに対し、コマンド「right」の停止を送信します。

それによって、クレーンが回転中の場合、動作を停止します。

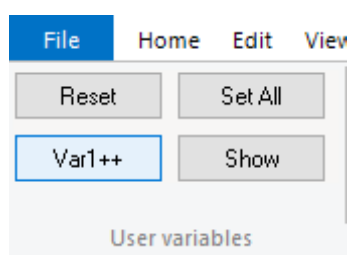
ボタン「Reset」をクリックすると、プロジェクトのすべての配置モデルに対し、コマンド「* RESET *」を送信します。

これにより、動作したモデルは元の状態に戻ります。

6.12.Sample_UserVariables

Sample_UserVariables はユーザー変数を取得、変更するスクリプトです。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「User variables」が追加されます。



ボタン「Reset」をクリックすると、ユーザー変数をクリアし、すべてのユーザー変数が 0 の状態となります。

ボタン「Var1++」をクリックすると、インデックス 1 のユーザー変数に 1 を加算します。

ボタン「Set All」をクリックすると、インデックス 0～511 のすべてのユーザー変数にインデックス番号と同じ値を設定します。

ボタン「Show」をクリックすると、インデックス 0～511 のユーザー変数の値をログファイルに出力します。（カレントディレクトリに出力されます）

ユーザー変数はシナリオのイベント遷移の条件で 사용할 ことが可能です。

シナリオのイベント遷移条件では表現できないような複雑な遷移条件でも、スクリプトによるユーザー変数制御とユーザー変数によるイベント遷移の条件によって、実現可能な場合があります。

6.13.Sample_VehiclePlacement

Sample_VehiclePlacement はシミュレーション上に新しい車両オブジェクトを生成するスクリプトです。

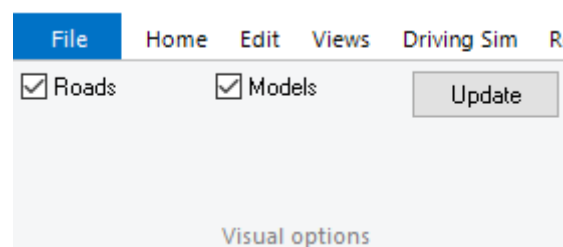
Mt Parkway.rd にて実行してください。

実行すると、道路「Nihondaira Park Way」の開始位置付近の、左側車線、右側車線の最も左の車線に新しい交通流の車が配置されます。

6.14.Sample_VisualOptionControl

Sample_VisualOptionControl は描画オプションを取得、制御するスクリプトです。

非同期チェックボックスを ON にして、実行するとリボンメニューのタブ「Python API Samples」にグループ「Visual options」が追加されます。



プロジェクト読み込み後や、描画オプション画面で変更した場合にチェックボックスのチェック状態の更新は行われません。

ボタン「Update」を押すことで更新され、現在の描画オプション状態がチェックボックスに反映されます。

チェックボックス「Roads」、「Models」のチェック状態を切り替えると道路、モデルの表示状態が切り替わります。

6.15.Sample_BrakeAssist

Sample_BrakeAssist は運転車両のアクセル・ブレーキのペダル入力进行操作して簡易的なアシストブレーキ動作をするスクリプトです。

運転車両と前方車両との車間距離が、予め定義している運転車両の速度に対する必要な車間距離を下回った場合に運転車両にブレーキをかけます。

運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行して下さい。アシストブレーキが動作している時は、リボン UI のユーザーフラグボタン「1」が押下状態（黒色）になります。

（今回のスクリプトでは交差点では車間距離の判定は行っていない）

6.16.Sample_ObstructiveDriving

Sample_ObstructiveDriving は運転車両の前方車両のステアリング入力、アクセル・ブレーキのペダル入力を操作し、簡易的な妨害運転動作を行うスクリプトです。

運転車両と前方車両の車間距離が、予め定義している運転車両の速度に対する必要な車間距離を下回った状態で一定時間経過（2 秒）すると、前方車両が蛇行運転と頻繁なブレーキによって妨害運転を行います。

運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行して下さい。妨害運転が動作している時は、リボン UI のユーザーフラグボタン「2」が押下状態（黒色）になります。

（今回のスクリプトでは交差点では車間距離の判定は行っていません）

6.17.Sample_DiagnosisDriving

Sample_DiagnosisDriving は運転車両の簡易的な運転診断をするスクリプトです。運転車両と前方車両との車間距離が、予め定義している運転車両の速度に対する必要な車間距離を下回るかを診断します。

「診断結果(var1)」「経過時間(var2)」、「速度(var3)」、「必要な車間距離(var4)」、「現在の車間距離(var5)」、「前方車両の ID(var6)」をユーザー変数に出力します。診断をしていない場合、各ユーザー変数は「-1」で出力します。ユーザー変数はログ出力プラグインにてログ出力することができます。

運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行して下さい。必要な車間距離を下回っている時は、リボン UI のユーザーフラグボタン「3」が押下状態（黒色）になります。

（今回のスクリプトでは交差点では車間距離の判定は行っていません）

ユーザー変数名	変数の意味	値の説明
Var1	診断結果	0:OK、1:NG、-1:非診断
Var2	経過時間(ms)	-1:非診断
Var3	速度(km/h)	-1:非診断
Var4	必要な車間距離(m)	-1:非診断
Var5	現在の車間距離(m)	-1:非診断
Var6	前方車両の ID	-1:非診断

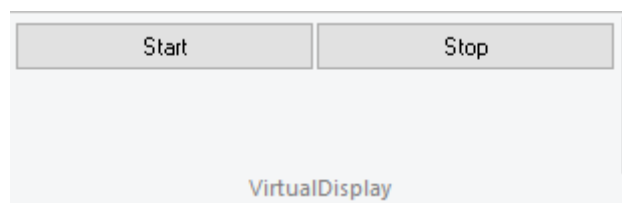
U	V	W	X	Y	Z
var 1	var 2	var 3	var 4	var 5	var 6
-1	-1	-1	-1	-1	-1
1	0	32.096	14	6.109683	32753255
1	0	32.096	14	6.109683	32753255
1	400.1058	26.54608	10	6.892321	32753255
1	749.9972	28.65553	10	8.114516	32753255
1	1152.677	30.1774	14	9.241935	32753255
1	1483.507	31.54796	14	10.18298	32753255
1	1816.763	33.12138	14	11.19318	32753255
1	1816.763	33.12138	14	11.19318	32753255
1	2183.42	34.35972	14	12.31394	32753255
1	2566.837	35.94196	14	13.18159	32753255
-1	-1	-1	-1	-1	-1
0	0	38.42177	14	14.92424	32753255
0	0	39.88333	14	15.58882	32753255
0	0	40.99616	14	16.15722	32753255
0	0	41.98672	14	16.54207	32753255
0	0	43.41881	14	16.83309	32753255

6.18. Sample_VirtualDisplay

Sample_VirtualDisplay はバーチャルディスプレイへスクリプトから OpenGL で図形を描画するスクリプトです。UC-win/Road で任意のバーチャルディスプレイを設定し、運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行して下さい。

リボンメニューのタブ「Python API Samples」のグループ「Virtual Display」の「Start」ボタンを押下するとバーチャルディスプレイへ車両速度に応じた線分を描画します。「Stop」ボタンを押下すると元のバーチャルディスプレイの描画へ戻ります。

注意：外部ツールからの実行では OpenGL 描画はできません。UC-win/Road からスクリプトを実行してください。



6.19. Sample GPSroads

Sample_GPSroads は Sample_VirtualDisplayDraw を利用して車の位置情報とミニマップ表示するプラグインです。UC-win/Road で任意のバーチャルディスプレイを設定し、運転走行を開始してから、非同期チェックボックスを ON にして、スクリプトを実行して下さい。

リボンメニューのタブ「Python API Samples」のグループ

「Sample_GPSroads」の「Start」ボタンを押下するとバーチャルディスプレイへ車両と道路を描画します。「Stop」ボタンを押下すると元のバーチャルディスプレイの描画へ戻ります。

表示されている車両の位置情報およびミニマップは、キーボード操作により表示内容を移動したり、拡大・縮小することができます。

対応するキーは以下の通りです。

「Home」 Key : Up

「End」 Key : Down

「Delete」 Key : Left

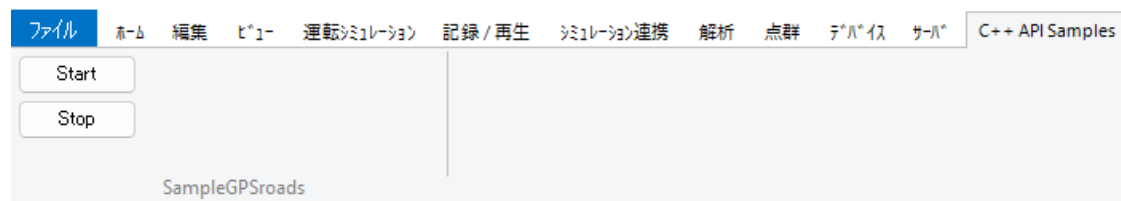
「Page Down」 Key : Right

「Insert」 Key : Zoom In

「Page Up」 Key : Zoom Out

複数のバーチャルディスプレイが存在する場合も対応しますが、キーボードによるコントロールは全バーチャルディスプレイを同時に操作されます。

注意：外部ツールからの実行では OpenGL 描画はできません。UC-win/Road からスクリプトを実行してください。



以上